SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3717**

COURSE OUTCOME COVERED **A.V.Rama krishna Reddy(192311123)**

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random	3
	tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves	3

efficiency.

Code of Conduct:

*I **A.V.Rama Krishna Reddy(192311123)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ramakrishna

Answers

1. Testing as an Engineering Activity

- **Testing should follow a planned and systematic process instead of random testing.**
- **Test cases must cover requirements, positive inputs, negative inputs, and boundary values.**
- **This improves defect detection, software quality, and reliability.**

2. Testing Process in Banking Systems

- **A defined testing process includes planning, test design, execution, and defect reporting.**
- **Banking functions such as deposits, withdrawals, and transfers must be tested carefully.**
- **Structured testing reduces transaction failures and improves security and accuracy.**

3. Testing as a Structured SDLC Process

- **Testing should be performed during requirements, design, coding, integration, and maintenance.**
- **Each phase uses suitable testing methods to identify defects early.**
- **This improves software quality and reduces development costs.**

4. Error, Defect, and Failure

- **An error is a human mistake, and a defect is a fault in the software.**
- **A failure occurs when the software produces an incorrect result.**
- **Example: An incorrect password condition may allow unauthorized login.**

5. Early Defect Detection

- **Early testing identifies defects during the initial stages of development.**
- **Requirement reviews and unit testing help detect problems before deployment.**
- **It reduces correction costs and improves software quality.**

6. Tester's Role in Requirement Analysis

- **Testers review requirements to identify ambiguity, missing details, and conflicts.**
- **They prepare test conditions and clarify doubts with developers and customers.**
- **Early involvement prevents defects and reduces rework.**

7. Origin of Defects

- **Defects can originate from unclear requirements, design mistakes, or coding errors.**

- **Incomplete payroll requirements may cause incorrect salary calculations.**
- **Clear requirements and regular reviews help prevent such defects.**

8. Boundary Value Analysis

- **Boundary Value Analysis tests values at and around input limits.**
- **For an age range of 18–60, test 17, 18, 19, 59, 60, and 61.**
- **It detects boundary-related defects and improves test coverage.**

9. Defect Repository

- **A defect repository stores defect descriptions, causes, severity, and solutions.**
- **Testers use historical defects to design better test cases.**
- **It reduces repeated defects and improves regression testing.**

10. ATM Functional Defect

- **A functional defect occurs when software does not behave according to requirements.**
- **Example: An ATM deducts money but fails to dispense cash.**
- **Transaction testing and failure-recovery testing help prevent this defect.**

11. Defect Clustering

- **Defect clustering means that many defects are concentrated in a few modules.**
- **Testers should prioritize modules with high defect counts and complexity.**
- **This improves testing efficiency and defect detection.**

12. Performance Defect

- **A performance defect occurs when software responds slowly under heavy user load.**
- **Load testing and stress testing help identify performance problems.**
- **Fixing bottlenecks improves response time and system reliability.**

13. Historical Defect Data

- **Historical defect data identifies modules and functions that failed previously.**
- **Testers reuse earlier test cases during regression testing.**
- **This improves coverage and saves testing time.**

14. Tester's Role in Integration Testing

- **Testers verify communication and data exchange between software modules.**
- **Early collaboration helps identify interface and integration defects.**
- **It improves data accuracy and system reliability.**

15. Defect Prevention

- **Defect prevention identifies and removes the root causes of recurring defects.**

- **Code reviews, clear requirements, and automated testing help prevent defects.**
- **It reduces rework, maintenance costs, and repeated failures.**

16. Standardized Mobile Application Testing

- **Testing should follow common procedures and documented test cases.**
- **Mobile apps must be tested across devices, operating systems, and network conditions.**
- **Standardization improves reliability and user satisfaction.**

17. Testing vs Debugging

- **Testing identifies defects by comparing actual results with expected results.**
- **Debugging finds the cause of a defect and corrects it.**
- **Both activities help improve software quality and reliability.**

18. Cross-Browser Testing

- **Cross-browser testing checks whether a website works correctly in different browsers.**
- **Test layouts, buttons, forms, and payments in Chrome, Firefox, Edge, and Safari.**
- **It improves compatibility and provides a consistent user experience.**

19. Validation in Logistics Applications

- **Input validation checks whether entered data meets defined rules.**
- **Testers should check missing fields, invalid dates, and incorrect package weights.**
- **Proper validation prevents incorrect processing and improves reliability.**

20. Systematic Test Case Design

- **Test cases should be designed from requirements instead of executing random tests.**
- **Include positive, negative, and boundary test scenarios with expected results.**
- **Systematic testing improves defect detection and test coverage.**

21. Data Consistency in Healthcare Systems

- **Data inconsistency occurs when connected modules display different information.**
- **Testers verify data mapping, transfer, synchronization, and transaction handling.**
- **Integration testing ensures accurate and consistent patient information.**

22. Tester's Role Across SDLC

- **Testers review requirements, examine designs, and prepare test cases during development.**
- **They perform integration, system, acceptance, and regression testing.**
- **Early involvement reduces defects, rework, and project delays.**

23. Testing During Peak Usage

- Peak-usage failures occur when systems cannot handle heavy user traffic.
- Load and stress testing identify capacity and performance problems.
- Better test coverage improves system stability and prevents failures.

24. Defect Documentation

- A defect report contains the ID, description, steps, severity, and expected results.
- Testers track defects until developers fix them and retesting is completed.
- Proper documentation improves communication and defect management.

25. Banking System Testing

- Banking systems require systematic testing of deposits, withdrawals, and transfers.
- Test insufficient funds, duplicate transactions, and network failures.
- Structured testing improves transaction accuracy, security, and reliability.

26. Edge Case Testing

- Edge case testing checks unusual inputs and values near permitted limits.
- For marks from 0 to 100, test 0, 100, -1, and 101.
- Boundary testing detects hidden defects and improves software reliability.

27. Process-Based Testing

- Process-based testing follows standardized procedures across software modules.
- Common test plans, test cases, and defect reports ensure consistency.
- Standardization improves quality and reduces missed defects.

28. Input Validation

- Input validation ensures that user data follows the required format and rules.
- Test empty fields, invalid values, special characters, and excessively long inputs.
- Proper validation improves software robustness, security, and reliability.

29. Defect Leakage

- Defect leakage occurs when defects escape testing and reach production.
- Requirement reviews, regression testing, and realistic test scenarios reduce leakage.
- These practices improve product quality and customer satisfaction.

30. Early Defect Detection Strategy

- Early defect detection identifies problems during requirements, design, and coding.
- Reviews and unit testing help find defects before they reach later stages.

- **This saves time, reduces costs, and improves software quality.**

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and wellorganized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand